Project Report

ERC-4337 Account Abstraction Security Testing: Automated Invariant Validation & AI-Driven Attack Generation

Team: Project Team 15

Course: CS6290 Privacy-Enhancing Technologies

Date: 2026/3/29

Repository: https://github.com/ZXCandZZZ/ERC-4337

Abstract

This project presents a systematic security evaluation of an ERC-4337 compatible smart wallet system. We combine specification-driven invariant testing (using Foundry) with AI-generated attack vectors to uncover vulnerabilities in the EntryPoint, wallet, and paymaster contracts. Our approach identified critical flaws, including acceptance of zero-signature UserOperations, nonce replay, and gas limit manipulation. We developed a complete testing pipeline with a Streamlit dashboard, generated over 1,200 attack variants, and validated fixes for high-severity issues. The results demonstrate the

effectiveness of combining formal invariants with AI-augmented fuzzing for blockchain security testing.

---

1. Introduction

Account Abstraction (ERC-4337) enables smart contract wallets without requiring externally owned accounts (EOAs). However, its complexity introduces new attack surfaces: signature validation bypasses, paymaster draining, nonce replays, and reentrancy. This project aims to:

- Define a clear threat model and security invariants for a minimal ERC-4337 system.
- Build an automated testing framework that combines invariant checks with AI-generated malformed UserOperations.
- Empirically measure the system's resilience and validate fixes for discovered vulnerabilities.

All code, scripts, and experimental data are available in the project repository.

---

2. Threat Model & Security Assumptions

2.1 System Components

- **EntryPoint** – singleton contract that validates and executes UserOperations.
- **Wallet (Account)** – contract implementing IAccount with a signature validation function.
- **Paymaster** – optional contract that sponsors gas (simplified version in our tests).
- **User** – submits signed operations via a bundler (simulated locally).

2.2 Assets

- User funds held in the wallet contract.
- Paymaster deposits locked in the EntryPoint.
- Wallet ownership (control over the account).
- Computational gas (paid by sender or paymaster).

2.3 Attacker Capabilities

- Submit arbitrary UserOperations to the EntryPoint (local bundler simulation).
- Deploy malicious wallet or paymaster contracts.
- Observe and replay operations (mempool access simulated).
- **No** access to users' private keys (signature forgery is a vulnerability, not a given capability).

2.4 Trust Assumptions

- EntryPoint implementation is correct per specification (we test against it).
- Bundler relays operations faithfully (no front-running considered in scope).
- Off-chain private keys are secure.

2.5 Threat Model Evolution

Version	Key Additions
V1 (M1)	Basic threats: signature forgery, replay, gas depletion.
V2 (M2)	Refined with zero- signature acceptance (discovered in M1), reentrancy, paymaster deposit exhaustion, gas overflow, delegatecall abuse.

The full Threat Model V2 is available in the repository as Threat Model V2.pdf.

---

3. Methodology

3.1 Specification-Driven Development

We adopted a spec-first approach with the following artifacts:

- **UserOperation interface & data models** – defined in the contracts.
- **Assumptions & invariants** – documented in invariants.md (10+ invariants across correctness, security, economic categories).

- **Acceptance criteria** – expressed as Given/When/Then statements (e.g., "Given a UserOp with invalid signature, When handleOps is called, Then it must revert with AA24").
- **Threat → Test mapping** – each threat in V2 is linked to one or more invariant tests.

3.2 Testing Infrastructure

- **Local blockchain** – Anvil (Foundry) for deterministic testing.
- **Smart contracts** – custom ERC-4337 EntryPoint, Wallet, and Paymaster (simplified but vulnerability-rich).
- **Invariant tests** – Foundry (Solidity) covering:
  - Signature validity (SEC-01)
  - Nonce monotonicity (COR-01)
  - Non-negative deposit balance (ECO-02)
  - Reentrancy prevention (SEC-03)
- **AI attack generator** – Python script using DeepSeek API to produce malformed UserOperations (invalid signatures, oversized gas fields, malformed paymaster data, etc.).
- **Batch test runner** – `run_batch.py` executes hundreds of attacks and logs results.
- **Streamlit dashboard** – visualises test outcomes (English and Chinese versions).

3.3 AI Attack Generation Pipeline

- **Prompt design** – iterative refinement from prompt_v2_design.md.
- **Dataset** – attacks_dataset_50plus.json (initial)

  and ATTACK_DATASET_1200_CLEAN.md (final 1,200+ variants).
- **Error correction** – AI outputs were manually reviewed; invalid JSON or malformed operations were rejected and corrected (see AI Collaboration Appendix).

---

4. Experiments & Results

4.1 Milestone 1 – Foundation

Discoveries:

- **Zero-signature vulnerability** – EntryPoint accepted a 65-byte zero signature as valid (transaction hash c7b6ed25…). This violated invariant SEC-01.
- **Deployment scripts** – Role A's scripts were made portable (.env template added).

4.2 Milestone 2 – Invariant Testing & AI Generation

Invariant test results (Foundry):

Invariant ID	Description	Status
COR-01	Nonce strictly increases	✅ Pass
SEC-01	Only valid signatures accepted	❌ Fail (zero signature)
SEC-03	No reentrancy during validation	✅ Pass (with malicious wallet)
ECO-02	Deposit balance never negative	✅ Pass

AI attack generation:

- Generated 50+ malformed UserOperations in M2.
- 12% initially contained invalid signatures (rejected by EntryPoint as expected).
- 3% triggered unexpected revert reasons (e.g., out-of-gas, array bounds) – logged for further analysis.

4.3 Milestone 3 – Full Validation & Fixes

High-severity attack vector fixed (2026/2):

- The zero-signature issue was patched by adding a check in EntryPoint: `require(signature.length == 65 && signature != bytes32(0))`.
- **Gas comparison:**
  - Before fix: zero-signature ops cost ~45k gas.
  - After fix: same ops revert immediately, costing ~12k gas (saving 33k gas per malicious attempt).

Batch test results (1,200 attacks):

Attack Category	Count	Successfully Rejected	Unexpected Behaviour
Invalid signature	600	598 (99.7%)	2 (out- of- gas)
Gas limit overflow	200	200 (100%)	0
Malformed paymaster data	200	198 (99%)	2 (revert with invalid error)
Nonce replay	200	200 (100%)	0

Failure report: TEST_FAILURE_REPORT_20260326.md documents two edge cases where out-of-gas occurred instead of a clean revert – these are low-severity but logged for future hardening.

4.4 Performance & Portability

- **Portability test** – Role A's deployment scripts ran successfully on a fresh Ubuntu 22.04 VM after adding .env template (screenshot in docs/portability-test-log.png).
- **Streamlit dashboard** – real-time visualisation of pass/fail rates, gas consumption, and attack type distribution.

---

5. Deliverables

All deliverables are available in the repository:

- **Final report (this document)**
- **Presentation slides** (slides.pdf in root)
- **Repository link** – https://github.com/ZXCandZZZ/ERC-4337 (example)
- **Code & scripts** – Foundry tests, batch runners, AI generator, Streamlit app.
- **Invariant definitions** – invariants.md
- **Threat models** – Threat Model V1.pdf, Threat Model V2.pdf
- **Attack**

  datasets – ATTACK_DATASET_1200_CLEAN.md, attacks_dataset_50plus.json
- **Test logs** – logs/ directory, TEST_FAILURE_REPORT_20260326.md
- **Update logs** – UPDATE_LOG_M4_20260326.md, Change_log

---

6. AI Collaboration Appendix

As required by the course, we document our responsible AI usage.

Tools Used

- **DeepSeek** (chat & code generation) – primary AI assistant.
- **GitHub Copilot** – minor autocompletion (not substantial).

What AI Was Used For

Task	Extent
Brainstorming attack vectors	High
Generating malformed UserOperation JSON	High
Writing Foundry invariant test templates	Medium
Drafting sections of this report	Low (only structure)
Debugging Python scripts (run_batch.py)	Medium

Example Where AI Output Was Wrong/Misleading

Situation: When generating attack variants for paymaster data, AI suggested using a field "paymasterAndData": "0x" + "00"*100 to cause an out-of-gas condition.

Why it was wrong: The ERC-4337 specification requires paymasterAndData to be either 0x (no paymaster) or a valid address + 48 bytes of data. A long string of zeros is rejected during decode before gas accounting, so it never causes an out-of-gas scenario.

How we corrected it: We manually replaced that attack with a valid paymaster address followed by random data that passes length checks but triggers a revert in validatePaymasterUserOp. The corrected attack was added to the final dataset.

Example Where AI Output Was Adopted

Situation: When writing the Foundry test for signature validity, AI suggested using `vm.expectRevert(abi.encodeWithSignature("FailedOp(uint256,string)", 0, "AA24 signature error"))` instead of a generic revert.

Why we adopted it: The exact error string makes the test more robust and aligns with the official ERC-4337 error codes, ensuring we catch the correct failure mode.

---

7. Conclusion

We successfully built an automated security testing framework for ERC-4337 that combines invariant definitions, AI-generated attacks, and batch execution. The project:

- Identified and helped fix a high-severity zero-signature vulnerability.
- Validated that nonce, balance, and reentrancy invariants hold.
- Generated over 1,200 attack variants with 99.7% correct rejection rate.
- Produced a reproducible, portable testing environment with a visual dashboard.

Future work could extend the framework to cross-chain replay attacks, integration with formal verification tools, and real-time mempool monitoring.